Smart Gate v3: Operating Guide

1. Overview

This guide explains how to start, use, stop and maintain the Smart Gate v3 test bench. In automatic mode the gate cycles idle → PIR trigger → detection → decision → open or close → idle; in manual mode an operator moves it from the web dashboard. It is a development prototype, not a field-ready gate.

2. What you need

- The test bench with the Jetson Orin Nano, camera, PIR, L298N driver, stepper and Hall sensors wired as in the Electrical Wiring Diagram REV 0.4.
- The Jetson's DC power supply and a charged 6×AA battery pack.
- A monitor, keyboard and mouse for the Jetson, or another device on the same network for the dashboard.
- Access to the repository: https://github.com/martin213214/SmartGate-2026, branch `main`.

3. Safety considerations

- Never connect the L298N's 5 V output to the Jetson. It would back-feed the board.
- The battery pack has no switch. Remove one cell to isolate it.
- Connect the pack only after the Jetson is running, and isolate it before moving or rewiring anything.
- Do not connect a real gate until travel is calibrated and a fuse, power switch and Hall resistors are fitted.
- Keep hands clear of the gate arm while the system is running.

4. Start the system

The system **starts automatically when the Jetson boots.** `smartgate.service` runs the motor-pin boot script (`setup/sgate_boot.sh`), then starts `src/main/live_detection.py` inside the `smartgate` container. No terminal is needed.

1. Power on the Jetson and wait about 1–2 minutes for the desktop to load and the service to start.
2. Connect the battery pack by refitting the removed cell.
3. Open the live stream at http://localhost:8080/stream on the Jetson, or start the dashboard and open it from another device (Section 6).
4. If the gate does not respond, check the service and the motor pins:

```
systemctl status smartgate          # should say "active (running)"
journalctl -u smartgate -f          # live output from live_detection.py (Ctrl + C to stop watching)
```

Then check the motor pins (Section 5).

Setting up auto-boot on a fresh unit. Run once: `sudo ~/SmartGate/setup/systemd_service_setup.sh`. Only use the repo's copy of this script. Older copies generate a service without the pinmux step, and the gate stops moving.

Running it by hand (development only). The service and a manual run both use port 8080, so stop the service first:

1. `sudo systemctl stop smartgate`
2. Start the container: `sgate`
3. Inside the container: `cd /app` then `python3 src/main/live_detection.py`
4. When finished, press Ctrl + C, `exit`, then `sudo systemctl start smartgate` to return to auto mode.

5. Check the motor pins

The boot script `setup/sgate_boot.sh` applies four motor-pin settings at start-up. If the gate does not move, read the four registers back:

```
for r in 0x243D020 0x243D010 0x2430068 0x2430070; do sudo busybox devmem $r; done
```

Header pin	Correct	Not applied
16	5	55
18	5	55
29	8	58
31	8	58

If you see 55, 55, 58, 58, run the boot script manually with `sudo ~/SmartGate/setup/sgate_boot.sh` and read the registers again.

6. Use the dashboard

The dashboard is the web app in `src/web-app`. It runs in its own Docker containers (the app plus a PostgreSQL database) on **port 8000**, separate from the gate service.

Start it by hand after each boot. This is deliberate. The Nano gets a new, random IP address every time it boots, and the web app needs that address to reach the Jetson. So after each boot:

1. On the Jetson, find its current address: `hostname -I`
2. Put that address in the web app's settings file, `src/web-app/.env`, on the `JETSON_LIVE_URL` line, for example `JETSON_LIVE_URL=http://192.168.0.42:8080`
3. Start the web app:

```
cd ~/SmartGate/src/web-app
sudo docker-compose up -d
```

If the address changes while the web app is running, edit `.env` again and rerun `sudo docker-compose up -d`.

Open it at http://smartgate.local:8000 from another device on the same network, or http://localhost:8000 on the Jetson. If the name does not resolve, use the address from `hostname -I` with port 8000.

Log in with GitHub. Users listed in `ADMIN_USERS` in the web app's `.env` file get the admin role, which is needed to move the gate. With `ENV=development`, a DUMMY LOGIN button is shown for testing; use `ENV=production` on the handover unit.

Page	What it shows
Dashboard (`stats.html`)	Animals detected, endangered and species counts, last detection, detections over time, detections by species, busiest times of day, gate opens and closes, and a searchable detection log
Live Stream (`gates.html`)	Live camera feed, gate status, Open / Close / Stop buttons (admins), Auto / Manual mode, snapshot, Jetson connection, latest detections, Hall and PIR readings, and gate analytics
Alerts	Detection alerts; admins can remove them
Users	User list; admins can change roles or remove users

Auto / Manual: Auto lets detection drive the gate; Manual stops the rules from moving it, so only the buttons do. The mode switch only works when the web app's `JETSON_LIVE_SERVICE` is set to `smartgate_nano`. With `http_server`, Open / Close / Stop still work but the mode buttons stay disabled.

The Jetson's own built-in page is still available at http://localhost:8080 (camera stream and open/close only).

7. Stop and power down

1. Isolate the battery pack by removing one cell.
2. Shut down the Jetson from the desktop or with `sudo shutdown now`. The service stops cleanly on shutdown, and `cleanup()` turns every GPIO pin off.

To stop the gate without shutting down: `sudo systemctl stop smartgate`. It starts again on the next boot, or with `sudo systemctl start smartgate`.

8. Configuration

The ruleset is in `config/config.json`.

Setting	Current value	Notes
Confidence threshold	0.6	Raised from 0.5 to stop a person being read as Fallow Deer
OPEN classes	Nine native species	Gate opens
CLOSE classes	Red Fox, Fallow Deer, Pig	Gate closes; any CLOSE class wins, even with a native animal in frame
Travel limit (`MAX_STEPS`, in `src/main/door_control.py`)	240	Placeholder; calibrate before connecting a real gate

Changes to which species open or close the gate are OpenSI's decision. Record every configuration change in the Change Register. Do not publish deployment-specific settings from `config.json` (for example a server token). The committed copy currently holds only the model, rules and port.

9. Maintenance and continuity

- **Container:** the `smartgate` image takes about two hours to rebuild. Do not run `docker system prune -a`, which deletes it.
- **Dashboard data:** detections, users and alerts are stored in the PostgreSQL volume `smartgate-data`. `docker-compose down -v` deletes it.
- **Code edits:** `~/SmartGate` is mounted as `/app`, so edits take effect after `sudo systemctl restart smartgate`, without a rebuild.
- **TensorRT engine:** rebuild it with `setup/convert_models.sh` after any JetPack upgrade or board change, because engines only run on the device and version they were built for.
- **Backups:** DC-motor and earlier versions of the controllers are kept as `io_control_old.py`, `door_control_old.py`, `io_control_dc.py` and `door_control_dc.py` in `src/main/`. DC mode can also be switched on with `MOTOR_MODE = 'dc'` in `io_control.py`.
- **Batteries:** replace the AA cells when the stepper slows or stalls.
- **Setup script:** only use the repo's `setup/systemd_service_setup.sh`. Older copies generate a service without the pinmux step, and the gate stops moving.

10. Test scripts

Script	What it tests
`src/main/stepper_test.py`	Stepper motion in both directions and the Hall stops
`src/main/detect_check.py`	Camera and detection only, without moving the gate
`src/test/get_pin.py`	Reads one GPIO pin
`src/test/set_pin.py`	Sets one GPIO pin
`test/GPIOtest.py`	GPIO access and pinmux diagnostics

11. Troubleshooting

Symptom	Likely cause	Fix
Gate does not move	Motor-pin settings not applied	Check registers (Section 5) and run the boot script
Gate does not move, registers correct	Battery pack isolated or flat	Refit the cell or replace the batteries
Gate stopped moving after re-running a setup script	Service regenerated without the pinmux step	Use the repo's `setup/systemd_service_setup.sh`, then check registers
Gate overruns or stops early	Magnet position or `MAX_STEPS` not calibrated	Check Hall stops with `stepper_test.py`
No camera image	Ribbon cable or overlay	Reseat the cable; check the IMX219 overlay is selected in `jetson-io`
`Address already in use` starting `live_detection.py`	`smartgate.service` is already running on port 8080	`sudo systemctl stop smartgate`, then start it again afterwards
"channel 7 is set to input" warning	Unwired fan pin still listed in `io_control.py`	Harmless; ignore
"engine built on a different device" warning	TensorRT engine metadata	Harmless if detection works; rebuild the engine if detection fails
Dashboard will not load	Web app not started after a reboot (it is started by hand because the IP address changes on every boot)	Follow Section 6: `hostname -I`, update `JETSON_LIVE_URL` in `.env`, then `sudo docker-compose up -d` in `src/web-app`
Dashboard shows the Jetson as disconnected	`JETSON_LIVE_URL` in the web app's `.env` still has the previous boot's IP address	Set it to the Jetson's address and port 8080, then `sudo docker-compose up -d` again
Person or object triggers the gate	Detection error	Report it with the frame; see Recommendation 4 (multi-frame check)

12. Known limitations

See Section 6.4 of the Final Delivery Report and the Limitations section of the README, including the uncalibrated travel limit, under-voltage motor supply and untested sustained operation.